

## **EXP 29 – Forward Chaining (Prolog)**

**Name: Hemalakshmi H (192571049)**

### **Aim**

To write a Prolog program that implements forward chaining to derive new facts from existing facts and rules until no further inference is possible.

### **Algorithm**

1. Declare dynamic facts using fact/1 so new facts can be added during execution.
2. Define initial facts such as has\_feathers, lays\_eggs, and can\_fly.
3. Define rules that specify when new facts can be inferred:
  - If an object has feathers and lays eggs → it is a bird.
  - If an object is a bird and can fly → it is a flying\_bird.
4. In forward\_chain:
  - Check if any rule can derive a new fact.
  - If yes, assert the new fact and print it.
  - Recursively continue chaining.
5. Stop when no more new facts can be derived.

### **Source Code**

```
prolog
```

```
:- dynamic(fact/1).
```

```
fact(has_feathers).
```

```
fact(lays_eggs).
```

```
fact(can_fly).
```

```
rule(bird) :-
```

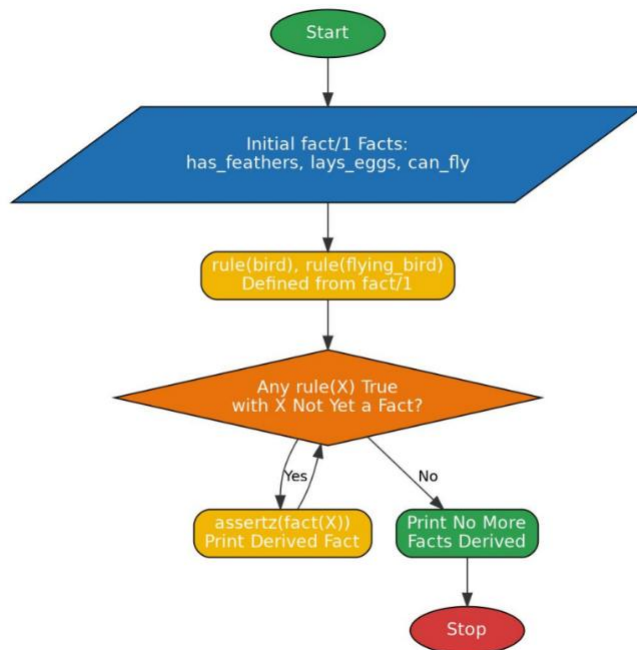
```
fact(has_feathers),  
fact(lays_eggs).
```

```
rule(flying_bird) :-  
    fact(bird),  
    fact(can_fly).
```

```
forward_chain :-  
    rule(X),  
    \+ fact(X),  
    !,  
    assertz(fact(X)),  
    format("Derived new fact: ~w~n", [X]),  
    forward_chain.
```

```
forward_chain :-  
    format("No more facts can be derived.~n").
```

## Flowchart:



## Output:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.9)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- cd('C:/Users/yanar/Documents/CSA1717').
true.

?- [forward].
true.

?- forward_chain.
Derived new fact: bird
Derived new fact: flying_bird
No more facts can be derived.
true.
```

## Result

The program successfully derives new facts such as *bird* and *flying\_bird* using forward chaining. It continues inference until no additional facts can be produced.